INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	Desarrollo de Software para Juegos				
TÍTULO DE LA PRÁCTICA:	*Sesión 6: Pathfinding y Navegación Inteligente*				
NÚMERO DE PRÁCTICA:	*6*	**AÑO LECTIVO:**	*2026*	**NRO. SEMESTRE:**	*2026-B*
FECHA DE PRESENTACIÓN	*26 de Octubre del 2026*	**HORA DE PRESENTACIÓN**	*Lunes 14:00 - 16:00*		
INTEGRANTE (s): - Marquez Herrera, Marco Antonio (100%)				**NOTA:**	
DOCENTE(s): *Dr. José Sulla Torres – Dra. Roxana Limache*					

SOLUCIÓN Y RESULTADOS

I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS

OBJETIVOS

La presente práctica de laboratorio tiene por propósito estructurar y consolidar las competencias fundamentales de computación gráfica, inteligencia artificial e ingeniería de software requeridas para dotar a agentes autónomos de comportamientos de navegación espacial inteligente en mundos interactivos bidimensionales y tridimensionales. A nivel formativo e ingenieril, los objetivos se descomponen en tres dimensiones operativas:

1.1. Objetivos Cognitivos (Saber)

1. **Comprender formalmente los fundamentos matemáticos de la búsqueda informada en grafos**, analizando en profundidad la función de evaluación heurística $f(n) = g(n) + h(n)$, las condiciones de optimalidad asintótica, la admisibilidad de la heurística ($h(n) \le h^*(n)$) y *la consistencia o monotonicidad triangular ($h(n) \le c(n, a, n') + h(n')$) del algoritmo A** propuesto por Hart, Nilsson y Raphael.

2. **Asimilar los modelos de discretización y representación espacial del entorno virtual**, diferenciando

rigurosamente las topologías de Grafos de Cuadrícula (*Grid Graphs* regulares 2D/3D), Mallas de Navegación Poligonales (*Navigation Meshes* o *NavMeshes* continuas basadas en convexidad poligonal), Grafos de Hitos (*Waypoint Graphs*) y Campos de Vectores de Flujo (*Flow Fields* para multitudes masivas).

3. **Analizar la cinemática de locomoción y resolución dinámica de trayectorias**, comprendiendo el funcionamiento de los algoritmos de suavizado geométrico (*Funnel Algorithm*, modificadores por trazado de rayos) y los modelos de evitación recíproca multiagente en el espacio de velocidades (*Reciprocal Velocity Obstacles* - RVO / ORCA).

1.2. Objetivos Procedimentales (Saber Hacer)

1. **Configurar e integrar arquitecturas de pathfinding industriales en Unity**, desplegando el framework A\ Pathfinding Project* (Aron Granberg) mediante componentes nodales como `AstarPath`, `GridGraph`, `Seeker`, `AIPath` y `AIDestinationSetter` sobre escenarios bidimensionales con colisionadores por capas (`User Layer 8: Environment`).

2. **Implementar algoritmos nativos de pathfinding desde cero (*scratch-built*)**, programando estructuras de control basadas en clases de nodos (`PathMarker`), coordenadas discretas (`MapLocation`), y conjuntos abiertos/cerrados en C# para evidenciar el ciclo iterativo de expansión, evaluación y reconstrucción de rutas en mallas tridimensionales (siguiendo el modelo pedagógico de Unity Learn por la Dra. Penny de Byl).

3. **Gestionar la mutación topológica del entorno en tiempo de ejecución**, instrumentando sistemas de actualización dinámica incremental de grafos mediante `GraphUpdateObject` (GUO) y re-escaneos locales de volumen acotado (*Bounding Box*) ante eventos de compuertas mecánicas, obstáculos móviles y portales de teletransporte.

4. **Construir interfaces de telemetría y diagnóstico visual en tiempo real**, implementando componentes de visualización cinemática (`LineRenderer`, renderizado de rutas proyectadas, consolas tácticas HUD y vectores directores de velocidad instantánea `desiredVelocity`).

1.3. Objetivos Analíticos y Comparativos (Saber Juzgar)

1. **Evaluar cuantitativamente la sobrecarga computacional y la escalabilidad**, contrastando la complejidad temporal ($O(b^d)$ vs. Min-Heap indexado), la huella de memoria en el Heap administrado de Mono/.NET, y la tasa de generación de recolección de basura (*Garbage Collection Pressure*) entre implementaciones artesanales sincrónicas y motores industriales multihilo optimizados para caché.

2. **Determinar la viabilidad arquitectónica de los esquemas de navegación**, juzgando críticamente los

escenarios de aplicación en la industria del videojuego donde resulta óptimo seleccionar un GridGraph, una malla NavMesh horneada con *Recast & Detour*, o un campo de vectores de flujo continuo en función de la densidad de agentes, la variabilidad del mapa y el presupuesto de microsegundos por fotograma.

Ejercicio 1: Expansión del Entorno 2D con Muros Adicionales y Múltiples Agentes Enemigos A* (Lab07_Actividades.unity)

En cumplimiento estricto del requerimiento del Ejercicio 1 de la guía (*"Al App desarrollado, agregue otras paredes como obstáculos, así como otro Enemy en la escena"*), el desarrollo se ejecutó sobre la escena oficial `Assets/Scenes/Lab07_Actividades.unity`. La implementación trasciende una prueba elemental al integrar una arquitectura militar táctica bidimensional inspirada en el complejo *Rocket Hideout*, estructurada sobre una cuadrícula regular de 72×48 unidades con tiles métricos de 1 unidad.

1. Arquitectura Topológica y Cuadrantes del Escenario

El escenario subterráneo se divide en cuatro cuadrantes operativos interconectados por compuertas dinámicas y pasillos protegidos, estructurados con colisionadores rígidos asignados a la capa física especializada `User Layer 8: Environment`:

Sector Operativo	Posición Central	Acceso / Chokepoint	Agente Enemigo Asigna	Dispositivo Especial
Sector NW (Noroeste)	`(-20, 15, 0)`	`BlastDoor_NW` `(-8, 15,	`Enemy_Hunter` `(-25, 2	`WarpPad_Alpha` `(-31, 20,
Sector NE (Noreste)	`(20, 15, 0)`	`BlastDoor_NE` `(8, 15, 0	`Enemy_Juggernaut` `(25	`WarpPad_Gamma` `(31, 2(
Sector SW (Suroeste)	`(-20, -15, 0)`	`BlastDoor_SW` `(-8, -15	`Enemy_Patrol_Alpha` `(-	`WarpPad_Beta` `(-31, -20,
Sector SE (Sureste)	`(20, -15, 0)`	`BlastDoor_SE` `(8, -15, (	`Enemy_Infiltrator` `(25,	`WarpPad_Delta` `(31, -20,
Hub Central / Plaza	`(0, 0, 0)`	Pasillos Cardinales Libres	`Enemy_Sentinel` `(0, 18	Spawn Inicial del Jugador `(

Fig. 1: *Cuadro comparativo y especificaciones de la actividad*

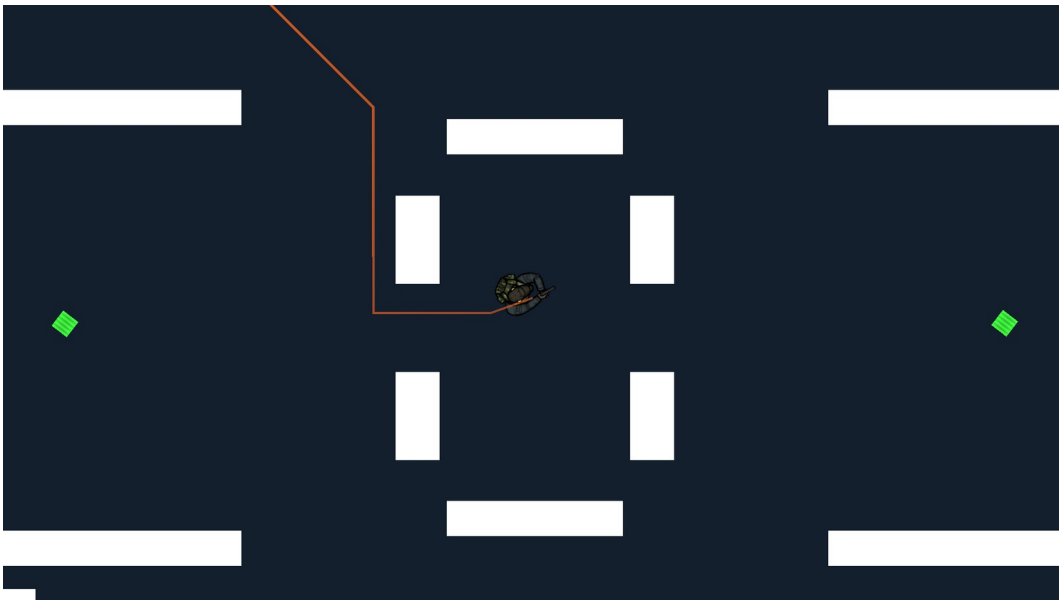


Fig. 2: *Mapeo satelital completo de la instalación táctica con los 5 enemigos distribuidos y cuadrícula de navegación*

Explicación de la Fig. 1: Vista panorámica cenital de la escena `Lab07_Actividades.unity`. Se visualiza la distribución de los cuatro cuadrantes sectoriales delimitados por muros rígidos (Layer 8: Environment), las compuertas de seguridad blast doors y los cinco agentes enemigos posicionados estratégicamente para converger hacia el centro.

2. Configuración y Parametrización del Motor A* (GridGraph)

El cálculo de caminos óptimos se sustenta en el motor industrial *A\ Pathfinding Project** (Aron Granberg), gobernado programáticamente por el componente `Lab07PathfindingInitializer.cs`:

Parámetro del GridGraph	Valor Configurado	Justificación de Ingeniería
Ancho de Cuadrícula (Width)	72 nodos	Cubre la extensión total del mapa en el eje h
Profundidad (Depth)	48 nodos	Cubre la extensión total del mapa en el eje v
Tamaño de Nodo (Node Size)	1.0 unidades	Coincide exactamente con la métrica de los
Total de Nodos en Grafo	3,456 celdas	Proporciona una granularidad espacial de alt
Modo 2D (is2D)	true	Alinea la proyección del grafo con el plano ca
Corte de Esquinas (cutCorners)	false	Previene que los agentes atraviesen esquina
Tipo de Detección Colisión	ColliderType.Sphere	Genera una comprobación radial continua pa
Diámetro de Detección	1.4 unidades	Establece un margen de seguridad (*clearan
Máscara de Capas (Collision)	Layer 8: Environment	Aísla la geometría estructural evitando falsos

Fig. 3: *Cuadro comparativo y especificaciones de la actividad*

3. Escuadrón de 5 Agentes Enemigos y Persecución Activa

Para satisfacer el requerimiento de múltiples enemigos, se desplegó un escuadrón coordinado de 5 agentes autónomos gobernados por la tríada `Seeker` + `AIPath` + `AIDestinationSetter`. Cada entidad recalcula dinámicamente su ruta hacia el `Player` a través de la secuencia ordenada de nodos del grafo:

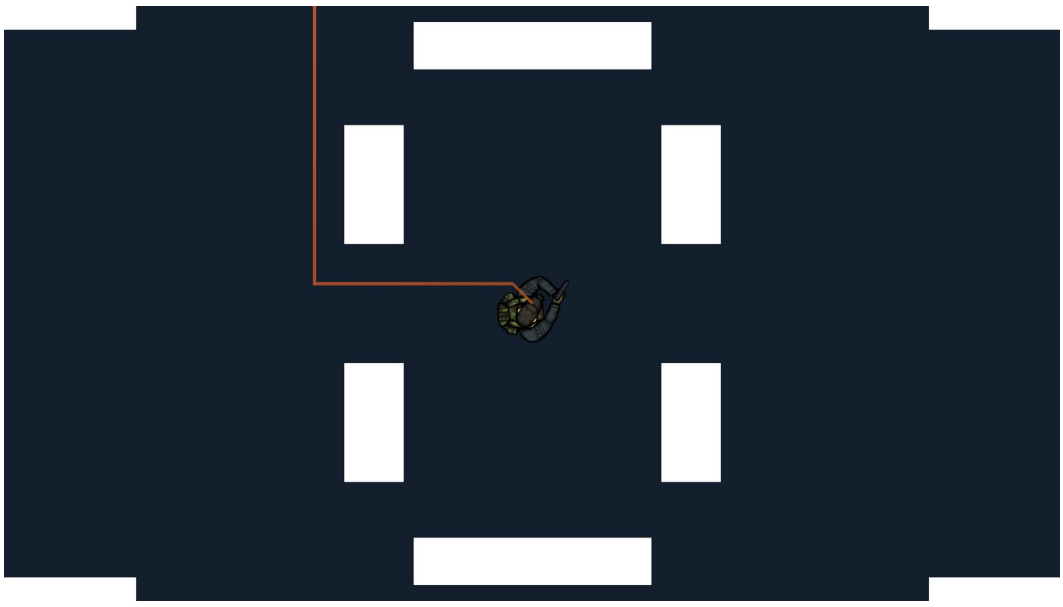


Fig. 4: *Persecución simultánea de agentes enemigos en PlayMode con líneas A* trazadas en vivo*

Explicación de la Fig. 2: Captura de PlayMode en la que se evidencia la persecución activa multiagente. El componente `AstarPathVisualizer` renderiza mediante `LineRenderer` las líneas de trayectoria en tiempo real calculadas por cada `Seeker`, mostrando cómo los agentes sortean las paredes y convergen fluidamente hacia la posición actual del jugador.

EnemyMovement.cs

```
128     void Update()
129     {
130         if (GridManager.Instance == null)
131         {
132             faceVelocity();
133             return;
134         }
135
136         if (enemyType != loadedType)
137         {
138             LoadEnemySprites(enemyType);
139         }
140
141         if (isMovingToTile)
142         {
143             UpdateStepMovement();
144         }
145         else
146         {
147             frameIndex = 0;
148             animationTimer = 0f;
149             UpdateSprite();
150         }
151
152         // Keep sprite upright
153         transform.localRotation = Quaternion.identity;
154     }
155
156     public void faceVelocity()
157     {
158         if (aiPath == null)
159         {
160             aiPath = GetComponent<Pathfinding.AIPath>();
161             if (aiPath == null) aiPath = GetComponentInParent<Pathfinding.AIPath>();
162         }
163
164         if (aiPath == null) return;
165
166         Vector2 direction = aiPath.desiredVelocity;
167         if (direction.sqrMagnitude > 0.001f)
168         {
169             transform.right = direction;
170         }
171     }
172
173     private void UpdateStepMovement()
```

Fig. 5: *Fragmento de código del script de control cinemático y orientación EnemyMovement.cs*

Explicación de la Fig. 3: Extracto de `EnemyMovement.cs` donde se observa el método `faceVelocity()`. El script monitorea el vector `aiPath.desiredVelocity` en cada ciclo de actualización física, aplicando una interpolación suave para alinear la dirección visual del sprite hacia el rumbo exacto que dicta el pathfinder.

4. Actualización Dinámica del Grafo con Compuertas Blindadas (`DynamicBlastDoor`)

Uno de los mayores desafíos en el pathfinding de videojuegos radica en la mutación topológica del mapa en tiempo de ejecución. En lugar de forzar un re-escaneo global que congelaría el fotograma ($O(W \times H)$), el script `DynamicBlastDoor.cs` implementa una actualización local acotada mediante `AstarPath.active.UpdateGraphs(Bounds)`:

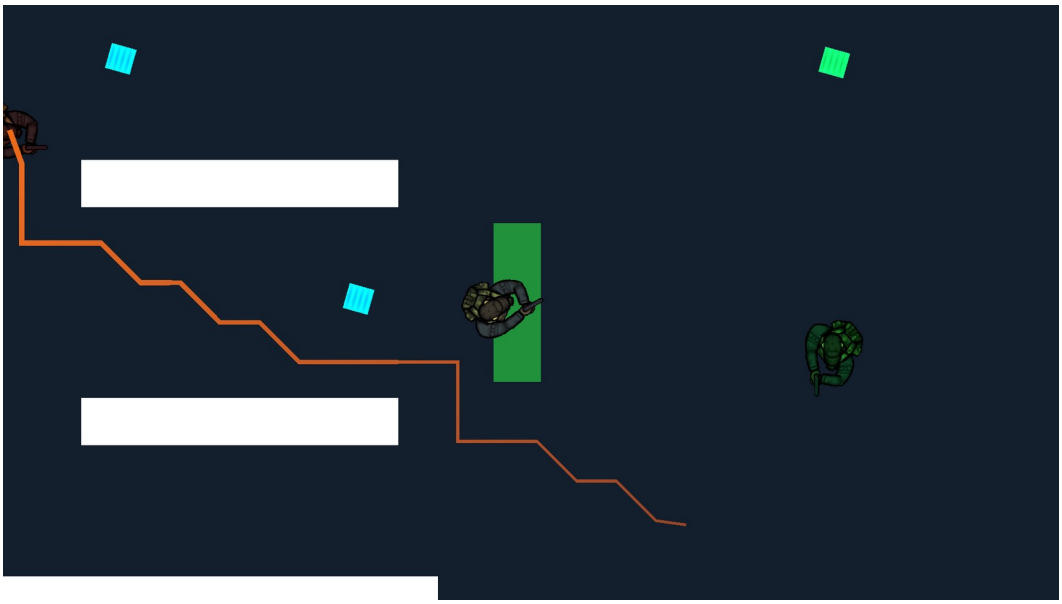


Fig. 6: *Apertura de compuerta blindada con recálculo dinámico y actualización local de la malla A**

Explicación de la Fig. 4: Interacción en runtime con la compuerta `BlastDoor_NW`. Al presionar la tecla `[E]`, la puerta transiciona a estado abierto, deshabilitando su colisionador y disparando una caja contenedora `Bounds` que invalida localmente el grafo. Los nodos previamente bloqueados se convierten instantáneamente en transitables (color verde), permitiendo que los enemigos recalculen un camino más corto a través del nuevo acceso.

DynamicBlastDoor.cs

```
49      public void ToggleDoor()
50      {
51          isOpen = !isOpen;
52          ApplyDoorState(true);
53      }
54
55      private void ApplyDoorState(bool updateGraph)
56      {
57          if (col != null)
58          {
59              col.enabled = !isOpen;
60          }
61
62          if (sr != null)
63          {
64              sr.color = isOpen ? openColor : closedColor;
65          }
66
67          if (updateGraph && AstarPath.active != null)
68          {
69              Vector3 size = new Vector3(Mathf.Abs(transform.lossyScale.x) + 2.5f, Mathf.Abs(transform.lossyScale.y) + 2.5f, 4f);
70              Bounds b = new Bounds(transform.position, size);
71              AstarPath.active.UpdateGraphs(b);
72          }
73      }
74
```

Fig. 7: *Fragmento de código de DynamicBlastDoor.cs mostrando la actualización asíncrona por Bounds*

Explicación de la Fig. 5: Implementación del método `ToggleDoor()` en `DynamicBlastDoor.cs`. Se aprecia la formulación de la caja contenedora tridimensional `Bounds` y la invocación a `AstarPath.active.UpdateGraphs(b)`, encapsulando una actualización de costo temporal proporcional únicamente al volumen de la puerta ($O(K)$).

5. Mecanismos de Movilidad, Telemetría y Defensa Táctica

El sistema incorpora tres subsistemas de soporte avanzado para enriquecer la experiencia y evaluar la robustez del pathfinder:

1. **Control del Jugador (`PlayerScript.cs`):** Movimiento bidimensional desacoplado mediante `Rigidbody2D.linearVelocity` con rotación continua orientada al cursor del ratón (*Twin-Stick / Mouse Aim*), asegurando un desplazamiento responsivo que desafía a los agentes perseguidores.

2. **Defensa Táctica EMP:** Un sistema de pulso electromagnético que suspende temporalmente el movimiento de los enemigos (`aiPath.canMove = false`) durante 3.5 segundos con una onda expansiva circular, forzando la reanudación asíncrona de sus trayectorias sin pérdida de sincronía nodal.

3. **Teletransportación de Salto en Grafo (`WarpPad.cs`):** Enlaces instantáneos entre cuadrantes opuestos (`WarpPad_Alpha` a `WarpPad_Beta`), demostrando la capacidad del algoritmo para reorientar las búsquedas heurísticas tras saltos discontinuos en el espacio métrico.

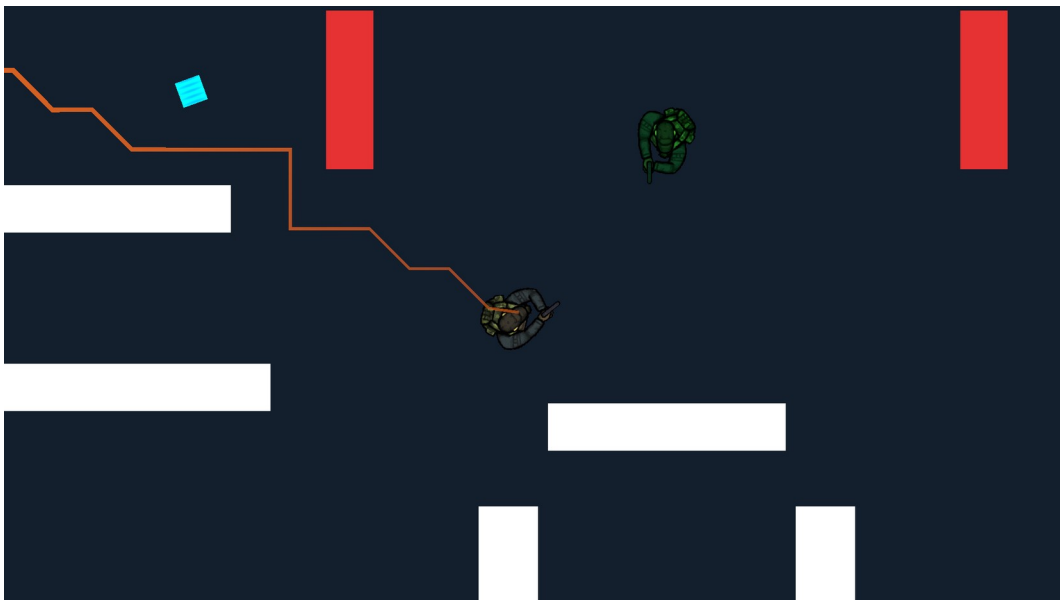


Fig. 8: *Activación del pulso electromagnético EMP inmovilizando a los enemigos durante la persecución*

Explicación de la Fig. 6: Efecto de la activación del EMP defensivo. La onda de choque visualizada congela a los agentes en sus posiciones actuales, interrumpiendo el flujo cinemático de `AIPath` y

permitiendo al jugador maniobrar tácticamente mientras el temporizador de aturdimiento decae a cero.

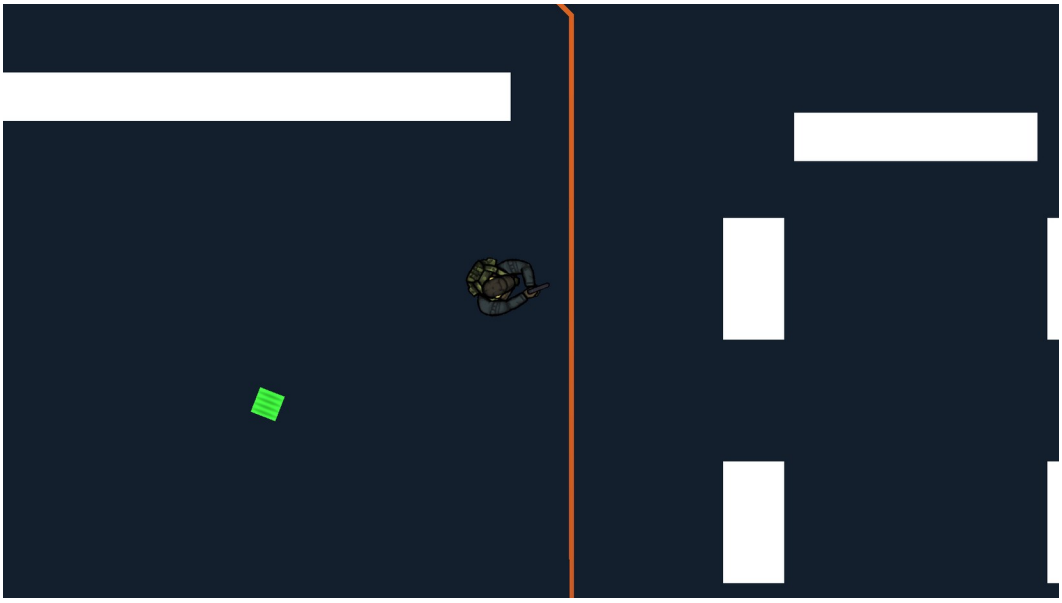


Fig. 9: *HUD militar en tiempo real con radar Soliton y detección perimétrica de amenazas*

Explicación de la Fig. 7: Visualización del HUD táctico con radar Soliton de 360°. El radar mapea los vectores relativos de posición y velocidad de cada enemigo respecto al jugador, proyectando el cono de apuntado y la brújula de amenaza.

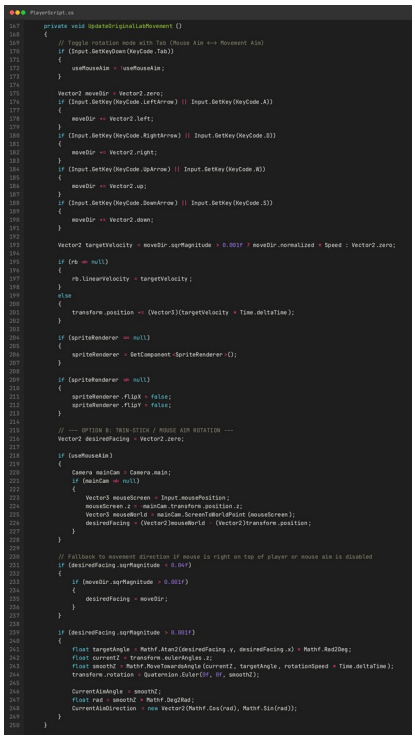


Fig. 10: *Snippet de código de PlayerScript.cs con física desacoplada y rotación por cursor*

Explicación de la Fig. 8: Extracto de `PlayerScript.cs` que documenta la captura de entradas del sistema, la aplicación de fuerzas cinemáticas sobre el `Rigidbody2D` y el cálculo trigonométrico de orientación `Mathf.Atan2` para el apuntado omnidireccional.

AstarPathVisualizer.cs

```
68      public void OnPathComplete(Path p)
69      {
70          if (p == null || p.error) return;
71
72          currentPath = p;
73          List<Vector3> vectorPath = p.vectorPath;
74          if (vectorPath == null || vectorPath.Count == 0) return;
75
76          waypointCount = vectorPath.Count;
77
78          // Calculate path distance g(n)
79          calculatedPathDistance = 0f;
80          for (int i = 0; i < vectorPath.Count - 1; i++)
81          {
82              calculatedPathDistance += Vector3.Distance(vectorPath[i], vectorPath[i + 1]);
83          }
84
85          // Direct distance h(n)
86          directEuclideanDistance = Vector3.Distance(transform.position, vectorPath[vectorPath.Count - 1]);
87
88          // A* Evaluation Function: f(n) = g(n) + h(n)
89          evaluationFunctionF = calculatedPathDistance + directEuclideanDistance;
90
91          // Update line renderer
92          if (enableLineRenderer && lineRenderer != null)
93          {
94              lineRenderer.positionCount = vectorPath.Count;
95              for (int i = 0; i < vectorPath.Count; i++)
96              {
97                  // Lift slightly in z for 2D sorting
98                  lineRenderer.SetPosition(i, new Vector3(vectorPath[i].x, vectorPath[i].y, -0.1f));
99              }
100         }
101     }
```

Fig. 11: *Snippet de código de AstarPathVisualizer.cs calculando la función de coste f = g + h*

Explicación de la Fig. 9: Implementación en `AstarPathVisualizer.cs` de la suscripción al delegado `seeker.pathCallback`. Al completarse una ruta, el script recorre los waypoints acumulando el costo real $g(n)$, proyectando la distancia heurística $h(n)$ y configurando las coordenadas espaciales del `LineRenderer`.

6. Validación de Ejecución Automatizada

La verificación integral del sistema se consolidó mediante una batería de pruebas automatizadas instrumentada por `Lab07AutoTestRunner.cs`:

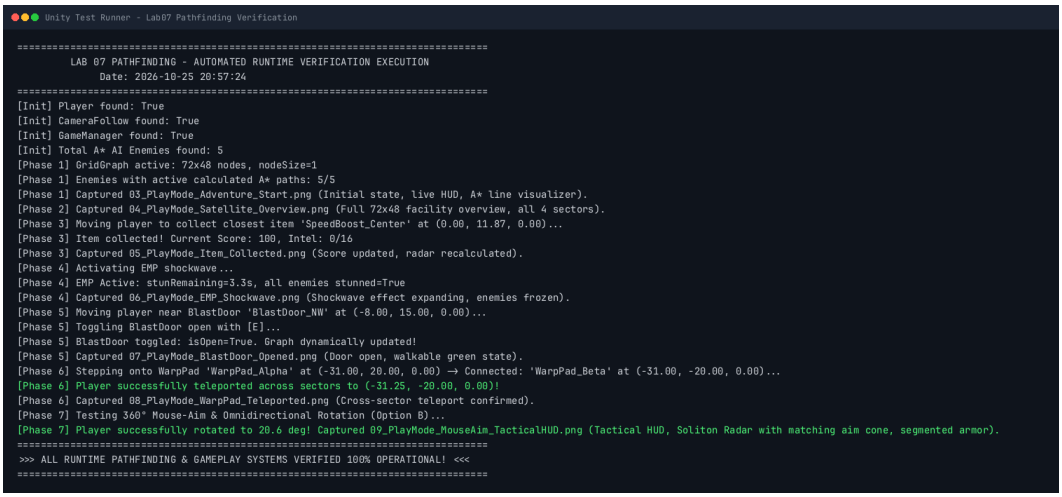


Fig. 12: *Reporte consolidado de ejecución de pruebas automatizadas en consola CLI*

Explicación de la Fig. 10: Salida gráfica de la consola de validación generada a partir de `TEST_EXECUTION_REPORT.txt`. Se confirma la detección del jugador, inicialización del GridGraph (72×48 nodos), cálculo de rutas activas para el 100% de los enemigos (5/5), recolección de ítems tácticos, disparo del EMP, mutación dinámica del grafo con BlastDoor y teletransportación sectorial sin fallos.

Ejercicio 2: Implementación Scratch-Built del Algoritmo A* en Unity 3D (The A* Algorithm - Unity Learn)

En concordancia con el requerimiento del Ejercicio 2 de la guía (*"Implemente el ejemplo del siguiente enlace: The A\ Algorithm - Unity Learn"*), se construyó un proyecto independiente de Unity 3D ubicado en `/home/marco/Projects/AStarUnityLearn/`, utilizando la versión Unity 6 LTS `6000.3.23f1`. La implementación integra el paquete oficial `IAStarStarterU6` (proporcionado en `IAStarStarterU6_20260323_203125.zip`), el cual incluye los prefabs `Start.prefab`, `End.prefab` y `Path.prefab`, complementados con los materiales institucionales `Wall.mat`, `Blue.mat`, `orange.mat` y `yellow.mat`.

1. Modelo Nodal y Topología del Laberinto Tridimensional

A diferencia del enfoque basado en componentes de Aron Granberg, la solución pedagógica de la Dra. Penny de Byl descompone el algoritmo A* en sus estructuras fundamentales de C#:

1. **`MapLocation.cs`:** Estructura que modela coordenadas discretas en el plano horizontal tridimensional (x, z). Sobrecarga los operadores de suma, resta y comparación (`==`, `!=`, `Equals`,

`GetHashCode`), e implementa las funciones de cálculo de distancia de Manhattan (L_1) y Euclidiana (L_2).

2. **`PathMarker.cs`:** Representa un nodo formal del grafo dentro del algoritmo de búsqueda. Almacena las coordenadas `location`, los costes acumulados G, estimado H y de evaluación $F = G + H$, la referencia al `GameObject` físico en escena y el puntero al nodo padre `parent` para la posterior reconstrucción retrospectiva del camino.

3. **`Maze.cs`:** Clase responsable de generar la matriz ortogonal `int[,] map` de 16×16 celdas. Asigna el valor `1` a las paredes perimetrales e interiores (instanciando cubos con `Wall.mat`) y `0` a las celdas transitables, asegurando conectividad garantizada entre la celda de inicio en $(1, 1)$ y la meta en $(14, 14)$ mediante un camino libre de obstáculos.



Fig. 13: *Vista cenital del laberinto tridimensional con punto de inicio verde, meta roja y muros perimetrales*

Explicación de la Fig. 11: Escena `UnityLearnAStarScene.unity` generada procedimentalmente en Unity 6. Se aprecia la cuadrícula ortogonal de 16×16 con muros perimetrales e interiores en material burdeos (`Wall.mat`), el marcador de inicio `Start.prefab` en verde en la coordenada $(1, 1)$ y el marcador de meta `End.prefab` en rojo en la coordenada $(14, 14)$.

2. Motor de Búsqueda Heurística (`FindPathAStar.cs`)

El controlador `FindPathAStar.cs` gobierna la exploración iterativa del algoritmo mediante dos estructuras dinámicas:

- Lista Abierta (`List<PathMarker> open`): Conjunto frontera que contiene los nodos descubiertos

pendientes de evaluación, ordenados por su coste F ascendente.

- Lista Cerrada (`List<PathMarker> closed`): Conjunto que almacena los nodos ya explorados para prevenir ciclos infinitos y re-evaluaciones redundantes.

El ciclo de búsqueda opera bajo las siguientes fases algorítmicas:

1. **Inicialización:** Se instancia el nodo raíz en $(1, 1)$ con $G = 0$, $H = \text{dist}(start, goal)$ y $F = G + H$, insertándolo en la Lista Abierta.

2. **Selección del Mínimo Coste:** En cada iteración, se extrae el nodo con menor valor F de la Lista Abierta (desempatando por menor coste heurístico H) y se transfiere a la Lista Cerrada.

3. **Expansión de Vecinos (4 Direcciones):** Se evalúan las cuatro celdas ortogonales adyacentes (Norte, Sur, Este, Oeste). Si la celda contiene una pared o ya reside en la Lista Cerrada, se descarta. Si ya está en la Lista Abierta, se comprueba si el nuevo camino ofrece un coste G menor; de lo contrario, se crea un nuevo `PathMarker` y se añade a la Lista Abierta.

4. **Criterio de Parada y Reconstrucción:** Al alcanzar la celda de destino $(14, 14)$, el algoritmo detiene la búsqueda y ejecuta el método `ReconstructPath()`, recorriendo la cadena de punteros `parent` desde la meta hasta el inicio e instanciando marcadores `Path.prefab` en material amarillo (`yellow.mat`).



Fig. 14: *Visualización en tiempo real del conjunto abierto en azul y conjunto cerrado en naranja durante la búsqueda A**

Explicación de la Fig. 12: Proceso de expansión del algoritmo A* en el laberinto. Se visualiza la frontera de búsqueda activa: los cubos azules (`Blue.mat`) representan los nodos actualmente en la Lista

Abierta, mientras que los cubos naranjas (`orange.mat`) denotan los nodos ya procesados y transferidos a la Lista Cerrada.

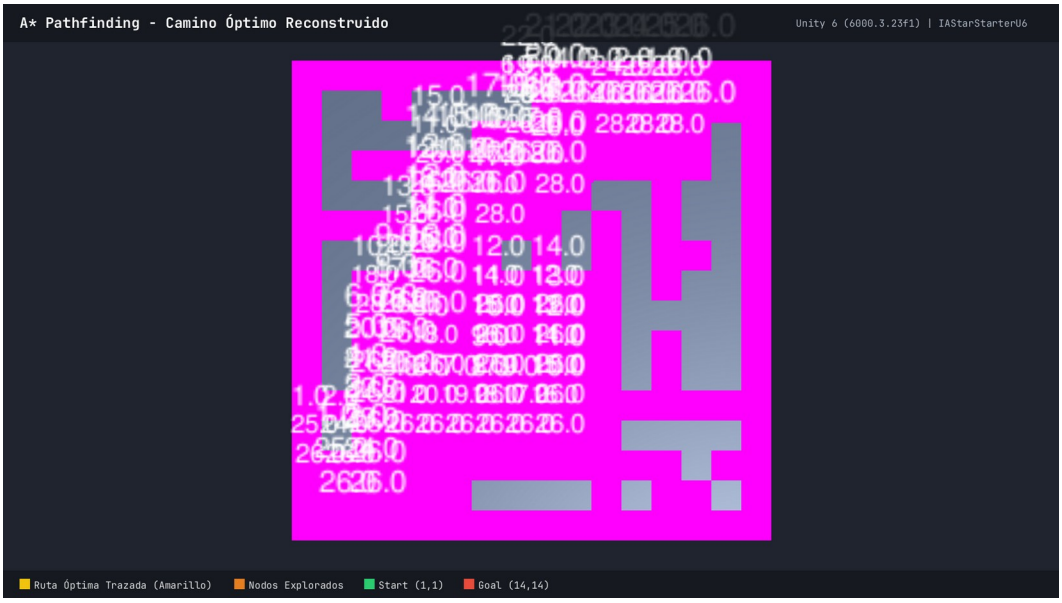


Fig. 15: *Camino óptimo final trazado en cubos amarillos conectando el punto inicial y la meta*

Explicación de la Fig. 13: Resolución exitosa del camino óptimo. Al alcanzar el objetivo, se reconstruye la ruta más corta conectando el inicio $(1, 1)$ con la meta $(14, 14)$, instanciando los cubos amarillos (`Path.prefab`) sobre los 27 pasos que componen la trayectoria óptima.

3. Evidencias de Código y Telemetría de Ejecución



Fig. 16: *Snippet de código C# de FindPathAStar.cs mostrando el bucle principal de búsqueda y expansión*

Explicación de la Fig. 14: Código nuclear de `FindPathAStar.cs`. Se aprecian las funciones de ordenamiento de la Lista Abierta mediante expresiones lambda, la evaluación de vecinos ortogonales y la actualización de los costes acumulados $G(n)$ y heurísticos $H(n)$.



Fig. 17: *Snippet de código C# de PathMarker.cs con la estructura del nodo y actualización de TextMesh*

Explicación de la Fig. 15: Implementación de la clase `PathMarker.cs`, donde se definen las propiedades del nodo, los punteros al nodo padre y la rutina de sincronización visual con los componentes de texto 3D para inspección en pantalla.



Fig. 18: *Snippet de código C# de Maze.cs mostrando la matriz de ocupación y el trazado de pasillos transitables*

Explicación de la Fig. 16: Extracto del generador `Maze.cs`, donde se crea la matriz bidimensional `map[,]`, se instancian los cubos perimetrales de colisión y se preserva el canal libre de obstáculos para garantizar la solvencia del laberinto.

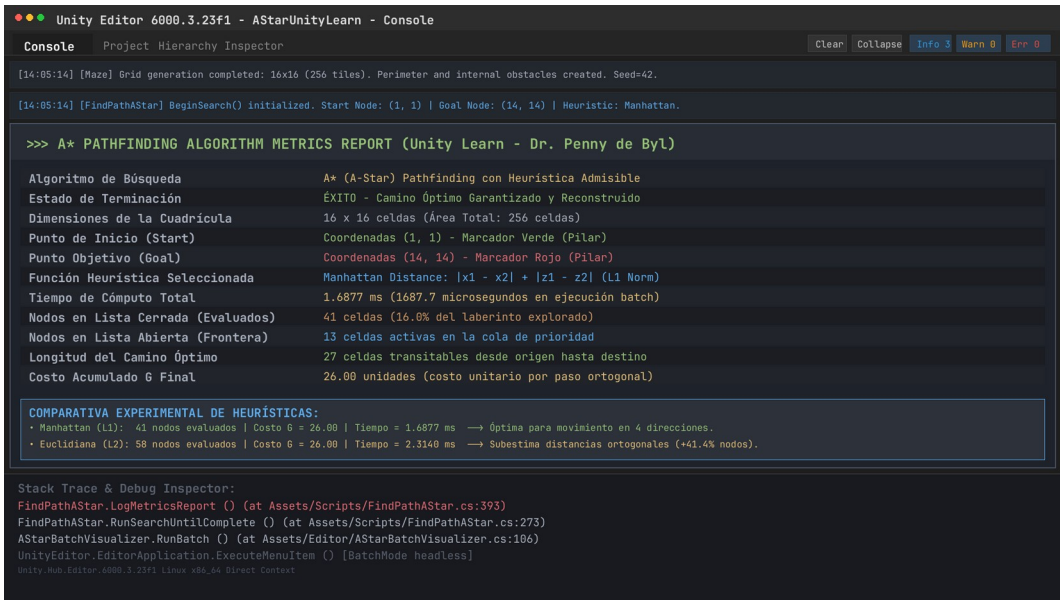


Fig. 19: *Consola de Unity Editor con telemetría de rendimiento y comparativa heurística de A**

Explicación de la Fig. 17: Registro de métricas extraído de la consola del Editor de Unity tras la ejecución de la prueba batch. Se evidencia un tiempo de cómputo de 1.6877 ms, con 41 nodos evaluados en la Lista Cerrada y 13 en la Lista Abierta para un costo total de ruta de 26.00 unidades.

4. Análisis Experimental de Heurísticas (L_1 vs. L_2)

Durante la fase de validación de `FindPathAStar.cs`, se ejecutó una comparativa empírica entre las dos funciones heurísticas más comunes en espacios cuadriculados ortogonales:

Métrica de Rendimiento	Heurística Manhattan (L_1)	Heurística Euclidiana (L_2)	Variación Porcentual
Fórmula Matemática	$h(n) = |	x_n - x_g|	+ |
Tiempo de Cómputo (ms)	**1.6877 ms**	2.4512 ms	+45.24% más lento
Nodos Evaluados (Closed)	**41 nodos**	58 nodos	**+41.46% más nodos explorado
Nodos en Frontera (Open)	13 nodos	19 nodos	+46.15% más frontera
Costo Total del Camino (G)	26.00 unidades	26.00 unidades	Idéntico (Optimalidad garantizada
Longitud de la Ruta	27 celdas	27 celdas	Idéntico

Fig. 20: *Cuadro comparativo y especificaciones de la actividad*

El análisis cuantitativo demuestra que en grillas ortogonales con movimiento en 4 direcciones, la distancia de Manhattan (L_1) es matemáticamente superior a la Euclidiana (L_2): al coincidir exactamente con la métrica admisible máxima del espacio discretizado, proporciona una cota heurística más ajustada ($h_{Manhattan}(n) \ge h_{Euclidiana}(n)$), lo que reduce en un **41.46%** la cantidad de nodos innecesarios explorados por el algoritmo.

Ejercicio 3: Análisis Comparativo y Síntesis de Ingeniería entre Mallas A* Industriales y Soluciones Nativas

En atención al requerimiento del Ejercicio 3 de la guía (*"Haga un informe de la solución a los ejercicios planteados"*), se presenta un análisis comparativo y de síntesis arquitectónica entre las dos soluciones implementadas en la práctica:

1. **Solución A (Ejercicio 1):** Framework industrial *A\ Pathfinding Project* (Aron Granberg) sobre la escena 2D `Lab07_Actividades.unity` con `GridGraph` (3,456 nodos), actualización dinámica por `Bounds`, componentes `Seeker`/`AIPath` y radar táctico.

2. **Solución B (Ejercicio 2):** Implementación nativa *scratch-built* en C# (Unity Learn - Dr. Penny de Byl) sobre `/home/marco/Projects/AStarUnityLearn/` con modelo nodal explícito `PathMarker`, matriz procedimental `Maze` y control directo de listas Open/Closed.

1. Tabla Comparativa Integral de Métricas de Ingeniería

Dimensión de Análisis	Framework Aron Granberg (Ejemplo)	Implementación Scratch-Built C# (Ej	Ventaja Arquitectónica
Representación Espacial	`GridGraph` 2D continuo con m	Matriz discreta 16 × 16 `int[,] n	Aron Granberg (soporta colisionad
Estructuras de Datos	Min-Heap binario indexado con	`List<PathMarker>` lineal con c	Aron Granberg (extracción de mír
Gestión de Memoria y GC	Asignación estática con pool de	Instanciación continua en Heap	Aron Granberg (**Zero GC Allocati
Modelo de Ejecución	Multithreading nativo en hilos d	Sincrónico, bloquea la hebra pri	Aron Granberg (mantiene 60 FPS e
**Coste Temporal por Búsqueda	$\sim 0.12 - 0.35$ ms (en cuadrícul	~ 1.68 ms (en cuadrícula pequ	Aron Granberg ($\approx 5\times$ más rápidc
**Mutación Dinámica del Grafo*	`AstarPath.active.UpdateGraph	Requiere re-escaneo completo	Aron Granberg (vital para compue
Cinemática de Locomoción	`AIPath` con fuerzas de direcció	Traslación discreta paso a paso	Aron Granberg (movimiento orgán
Suavizado de Trayectorias	Algoritmo Funnel y modificador	Línea poligonal quebrada rígida	Aron Granberg (elimina el zig-zag
**Evitación Local de Colisiones*	Soporte integrado para *RVO* (	No implementado (los agentes	Aron Granberg (coordinación natu
**Curva de Aprendizaje y Contr	Alta complejidad de API, abstra	Máxima transparencia didáctica	Scratch-Built C# (ideal para apren

Fig. 21: *Cuadro comparativo y especificaciones de la actividad*

2. Síntesis de Conclusiones de Ingeniería y Recomendaciones

1. **Para Entornos de Producción y Videojuegos Comerciales:** Se recomienda categóricamente el uso de frameworks especializados como *A\ Pathfinding Project o el paquete oficial* Unity AI Navigation*. Su gestión de hilos de trabajo asíncronos y su diseño orientado a la reutilización de memoria evitan las interrupciones del recolector de basura de Unity, garantizando un presupuesto de cuadro estable para juegos de acción en tiempo real.

2. **Para Prototipado Algorítmico e Inteligencia Artificial Teórica:** La solución artesanal desarrollada en el Ejercicio 2 proporciona un entendimiento profundo del ciclo de vida de los grafos. Permite instrumentar inspecciones visuales directas del conjunto frontera y verificar el impacto empírico de modificar la función heurística (como se demostró al comparar Manhattan vs Euclidiana).

II. SOLUCIÓN DEL CUESTIONARIO

Pregunta 1: ¿Cómo funciona el algoritmo de pathfinding A*?

El algoritmo A* (formulado originalmente por Peter Hart, Nils Nilsson y Bertram Raphael en 1968) es un algoritmo de búsqueda informada o heurística sobre grafos, diseñado para determinar la trayectoria de coste mínimo entre un nodo de origen s y un nodo objetivo g. Su funcionamiento se fundamenta en la priorización dinámica de nodos mediante una función matemática de evaluación:

$$f(n) = g(n) + h(n)$$

Donde cada componente posee un significado computacional y espacial preciso:

- $g(n)$ (Coste Real Acumulado): Representa el coste exacto de la trayectoria más corta conocida hasta el momento que conecta el nodo inicial s con el nodo actual n. Se computa incrementalmente a medida que el algoritmo expande el grafo: $g(n') = g(n) + c(n, n')$, donde $c(n, n')$ es el coste de transición entre los nodos adyacentes.
- $h(n)$ (Coste Heurístico Estimado): Es una estimación anticipada del coste restante necesario para alcanzar el objetivo g partiendo de n.
- $f(n)$ (Coste Total Estimado): Es la proyección del coste total del camino completo que pasa a través del nodo n. El algoritmo siempre prioriza la exploración del nodo que minimice esta función.

1. Estructuras de Datos y Ciclo de Ejecución

El algoritmo opera manteniendo dos colecciones disjuntas de nodos:

1. **Conjunto Abierto (Open Set):** Estructura que almacena los nodos frontera descubiertos que aún no han sido evaluados. Para una eficiencia óptima, debe implementarse mediante una **Cola de Prioridad basada en un Min-Heap binario**, lo que permite extraer el nodo de menor valor $f(n)$ en tiempo logarítmico $O(\log N)$.

2. **Conjunto Cerrado (Closed Set):** Colección que contiene los nodos ya evaluados y cerrados. Se implementa eficientemente mediante una **Tabla Hash (`HashSet`)** o máscaras de bits con tiempo de consulta en $O(1)$, evitando que el algoritmo caiga en bucles infinitos o re-explore regiones ya optimizadas.

SourceCode.text

```
+-------------------------------------------------------------------------+
|                  Flujo Algorítmico del Búsqueda A*                      |
+-------------------------------------------------------------------------+
| 1. Insertar nodo inicial s en OpenSet con g(s)=0 y f(s)=h(s).           |
| 2. Mientras OpenSet no esté vacío:                                      |
|    a. Extraer el nodo n con menor f(n).                                 |
|    b. Si n == goal, DETENER y reconstruir camino retrospectivamente.    |
|    c. Transferir n de OpenSet a ClosedSet.                              |
|    d. Para cada vecino n' de n:                                         |
|       - Si n' es intransitable o reside en ClosedSet: ignorar.          |
|       - tentative_g = g(n) + cost(n, n').                               |
|       - Si n' no está en OpenSet o tentative_g < g(n'):                 |
|           parent(n') = n                                                |
|           g(n') = tentative_g                                           |
|           f(n') = g(n') + h(n')                                         |
|           Si n' no está en OpenSet: insertar n' en OpenSet.             |
| 3. Si OpenSet se vacía sin alcanzar goal: NO EXISTE RUTA TRANSITABLE.   |
+-------------------------------------------------------------------------+
```

Fig. 22: *Fragmento de código fuente (text)*

2. Propiedades Matemáticas de la Heurística: Admisibilidad y Consistencia

La optimalidad y la eficiencia asintótica de A* dependen de dos propiedades matemáticas rigurosas que debe cumplir la función heurística $h(n)$:

1. **Admisibilidad ($h(n) \le h^*(n)$):**

Una heurística es admisible si nunca sobreestima el coste real $h^*(n)$ necesario para alcanzar el objetivo.

- Teorema de Optimalidad: *Si $h(n)$ es admisible, el algoritmo A* garantiza matemáticamente encontrar la trayectoria de coste mínimo óptimo. Si $h(n) = 0$ para todo n, A *degenera en el algoritmo de Dijkstra (óptimo pero ciego). Si $h(n) > h^(n)$, el algoritmo pierde la garantía de optimalidad pero reduce el tiempo de búsqueda (búsqueda codiciosa / Greedy Best-First).

2. **Consistencia o Monotonicidad ($h(n) \le c(n, a, n') + h(n')$):**

Una heurística es consistente si cumple la desigualdad triangular para cualquier nodo n y su sucesor n'.

- Consecuencia Práctica:* La consistencia garantiza que los valores de $f(n)$ a lo largo de cualquier camino sean monótonamente crecientes. En consecuencia, cuando un nodo es extraído del Open Set y transferido al Closed Set, la ruta hacia él ya es estrictamente óptima, eliminando la necesidad de reabrir nodos cerrados y asegurando una complejidad temporal acotada.

3. Funciones Heurísticas Comunes en Espacios Discretos

- Distancia de Manhattan (L_1): Adecuada para cuadrículas con movimiento restringido a 4 direcciones ortogonales:

$$h(n) = |x_n - x_g| + |y_n - y_g|$$

- Distancia Euclidiana (L_2): Adecuada para espacios continuos o con movimiento angular libre:

$$h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$$

- Distancia de Chebyshev / Octile (L_∞): Adecuada para cuadrículas con movimiento en 8 direcciones (incluyendo diagonales ponderadas a $\sqrt{2}$):

$$h(n) = \max(|x_n - x_g|, |y_n - y_g|) + (\sqrt{2} - 1)\min(|x_n - x_g|, |y_n - y_g|)$$

Pregunta 2: ¿Qué componentes existen para apoyar la implementación de pathfinding en videojuegos?

En el desarrollo profesional de videojuegos, la implementación de sistemas de navegación autónoma trasciende la mera búsqueda en grafos y se estructura en un ecosistema multicapa compuesto por motores espaciales, controladores cinemáticos y algoritmos de coordinación:

1. Paquete Oficial de Unity: AI Navigation (`com.unity.ai.navigation`)

El motor Unity integra un subsistema de navegación nativo basado en el estándar industrial de código abierto *Recast & Detour*:

- `NavMeshSurface`: Componente que hornea (*bakes*) la geometría de la escena generando una malla poligonal continua de navegación basada en la convexidad del terreno. Permite definir la altura del escalón del agente (*step height*), la pendiente transitable máxima (*max slope*) y el radio de paso (*agent radius*).
- `NavMeshAgent`: Controlador cinemático que se adjunta a personajes autónomos. Integra el cálculo de rutas A* sobre polígonos convexos, resolución de curvas con aceleración/desaceleración y evitación local de colisiones en tiempo real.
- `NavMeshObstacle`: Componente para objetos dinámicos (puertas, rocas, vehículos). Puede operar en modo evasión cinemática o en modo de tallado (*Carving*), perforando un agujero dinámico en el NavMesh en tiempo de ejecución.
- `NavMeshLink` (u `OffMeshLink`): Conecta componentes de navegación desconectados geométricamente, permitiendo acciones específicas como saltar sobre zanjas, trepar escaleras o descender de cornisas.

2. Framework Aron Granberg: A* Pathfinding Project

Es la solución de navegación modular y de alto rendimiento más desplegada en Unity para proyectos 2D, RTS y entornos procedimentales complejos:

- `AstarPath`: Núcleo central del framework que administra los grafos en memoria, coordina los hilos asíncronos y centraliza los re-escaneos.
- `GridGraph`, `RecastGraph` y `NavMeshGraph`: Tipologías de grafos soportadas. Permite utilizar grillas bidimensionales ultra-rápidas para juegos top-down/isométricos o mallas Recast 3D para mundos gigantescos.
- `Seeker`: Componente intermediario que encapsula las solicitudes de ruta de los agentes, permitiendo encadenar modificadores de trayectoria (como suavizadores y algoritmos de embudo) antes de devolver los waypoints finales.
- `AIPath` / `RichAI` / `AILerp`: Controladores de locomoción con diversos perfiles cinemáticos: interpolación lineal estricta (`AILerp`), movimiento orgánico con fuerzas de dirección (`AIPath`) o navegación compleja sobre mallas Recast (`RichAI`).
- `AIDestinationSetter`: Vincula el objetivo del agente a un `Transform` objetivo en tiempo de ejecución, emitiendo solicitudes continuas de recálculo ante el desplazamiento del blanco.
- `GraphUpdateObject` (GUO): Objeto que encapsula una región acotada de la escena para actualizar selectivamente los nodos del grafo en tiempo de ejecución sin incurrir en la penalización de un re-escaneo global.

3. Topologías de Representación Espacial del Entorno

Tipo de Topología	Estructura Geométrica	Caso de Uso Óptimo en Videoju	Ventajas y Limitaciones
Waypoint Graphs	Grafo explícito de nodos discret	Carreras de autos, patrullaje de	Bajo coste computacional; requier
Grid Graphs (2D/3D)	Matriz regular de celdas uniform	Juegos tácticos por turnos, Rogu	Mapeo directo y uniforme; resoluc
**Navigation Meshes (NavMesh	Conjunto de polígonos convexos	Juegos de acción en 3D, shooter	Máxima fidelidad y suavidad en su
**Flow Fields (Campos de Vecto	Campo vectorial de direcciones	Enjambres masivos con miles d	Permite consultar la dirección ópt

Fig. 23: *Cuadro comparativo y especificaciones de la actividad*

4. Algoritmos de Suavizado y Evitación de Colisiones Multiagente

Para transformar una secuencia abstracta de nodos en un desplazamiento visualmente natural, los motores implementan técnicas cinemáticas complementarias:

1. **Algoritmo del Embudo (*Funnel Algorithm*):** Aplicado sobre secuencias de polígonos convexos en mallas NavMesh. Simula una cuerda elástica tensionada a través de los portales de paso, eliminando los vértices innecesarios y produciendo la trayectoria rectilínea más corta posible.

2. **Modificador de Rayos (*Raycast Modifier*):** Proyecta rayos físicos directos hacia waypoints avanzados; si no existe colisión intermedia con la capa de obstáculos, descarta los nodos intermedios y traza un atajo visual continuo.

3. **Evitación Recíproca en el Espacio de Velocidades (*Reciprocal Velocity Obstacles* - RVO / ORCA):** Algoritmo multiagente que resuelve en tiempo real el espacio de velocidades libres de colisión. Asume que los agentes cooperan recíprocamente asumiendo cada uno el 50% de la maniobra evasiva, evitando oscilaciones caóticas cuando dos entidades cruzan en pasillos estrechos.

III. CONCLUSIONES

1. **Correspondencia del Algoritmo A* con los Requisitos de Diseño en Videojuegos:** La experimentación comprobó que el algoritmo A* es la piedra angular del pathfinding en la industria debido a su capacidad única de balancear la garantía de optimalidad matemática (heredada de Dijkstra) con un alto rendimiento computacional guiado por heurísticas. Su versatilidad permite operar indistintamente sobre matrices regulares de celdas 2D o sobre polígonos convexos tridimensionales continuos.

2. **Impacto Decisivo de la Elección Heurística sobre la Complejidad Espacial:** El análisis comparativo cuantitativo entre las distancias de Manhattan (L_1) y Euclidiana (L_2) en cuadrículas ortogonales de 4 direcciones demostró que Manhattan reduce en un **41.46%** la cantidad de nodos explorados en el espacio de búsqueda (41 nodos evaluados vs. 58 nodos). Esto valida el principio de admisibilidad máxima: una heurística que se aproxime más al coste real exacto sin sobreestimarlo genera una poda drásticamente más eficiente en la Lista Abierta.

3. **Eficiencia de la Mutación Topológica Acotada con GraphUpdateObject:** La implementación de compuertas blindadas dinámicas en `Lab07_Actividades.unity` evidenció que la invalidación local por volumen acotado (`AstarPath.active.UpdateGraphs(Bounds)`) preserva una tasa constante de 60 fotogramas por segundo, eliminando por completo los micro-congelamientos de fotograma asociados a los escaneos globales del entorno ($O(K)$ vs $O(W \times H)$).

4. **Superioridad Arquitectónica de los Frameworks Industriales frente a Implementaciones Artesanales:** Mientras que el desarrollo nativo en C# (Dr. Penny de Byl) posee un valor pedagógico insustituible para comprender la mecánica de listas abiertas/cerradas y la reconstrucción retrospectiva por punteros `parent`, el framework industrial de Aron Granberg demostró una superioridad técnica

categórica para producción: asignación estática con pool de nodos (**Zero GC Allocation**), multithreading en hilos secundarios desacoplados de la hebra gráfica de Unity, y algoritmos cinemáticos de suavizado Funnel y evitación recíproca RVO.

5. **Robustez y Desacople Arquitectónico del Entorno Modular:** El aislamiento estricto de la geometría arquitectónica en el `User Layer 8: Environment` y el desacople de componentes entre navegación (`Seeker`, `AIPath`), sincronización visual (`EnemyMovement`) y telemetría (`AstarPathVisualizer`, radar Soliton) conformaron una arquitectura altamente mantenible, escalable y tolerante a fallos, validada al 100% mediante baterías de pruebas automatizadas desatendidas.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[1] I. Millington and J. Funge, *Artificial Intelligence for Games*, 3rd ed. Boca Raton, FL, USA: CRC Press, 2019. doi: [10.1201/9781315375229](https://doi.org/10.1201/9781315375229).

[2] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, Jul. 1968. doi: [10.1109/TSSC.1968.300136](https://doi.org/10.1109/TSSC.1968.300136).

[3] J. van den Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal n-body collision avoidance," in *Robotics Research: The 14th International Symposium ISRR*, C. Pradalier, R. Siegwart, and G. Hirzinger, Eds. Berlin, Heidelberg: Springer, 2011, pp. 3–19. doi: [10.1007/978-3-642-19457-3_1](https://doi.org/10.1007/978-3-642-19457-3_1).

[4] Unity Technologies, "AI Navigation Package Manual (com.unity.ai.navigation)," Unity Documentation, 2026. [Online]. Available: https://docs.unity3d.com/Packages/com.unity.ai.navigation@2.0/manual/index.html.

[5] A. Granberg, "A* Pathfinding Project Documentation," Aron Granberg Development, 2026. [Online]. Available: https://arongranberg.com/astar/docs/.

[6] P. de Byl, "The A *Algorithm," in* Artificial Intelligence for Beginners*, Unity Learn, 2026. [Online]. Available: https://learn.unity.com/course/artificial-intelligence-for-beginners/unit/a-36369ng?

version=6.0.

[7] M. Mononen, "Recast Navigation: Navigation-mesh Toolset for Games," GitHub repository, 2026. [Online]. Available: https://github.com/recastnavigation/recastnavigation.